

Numerical Methods for Mathematical Finance

Matteo Garbelli
`matteo.garbelli@univr.it`

January 2026

1 Exam Procedures

- You can choose one project from the list in Section 3 (optionally you can propose your own idea). Every student should select a different project, so I recommend you consult first with your colleagues.
- The choice of the project needs to be approved. For this, send me an email.
- For passing the 6-credits course, the project consists of three components:
 1. A detailed **report**. You need to write a report in PDF where you explain the problem, the methodology used, derivation of formulas, etc. (it must be a self-contained report), and then of course the results found.
 2. The **code** developed for the project, properly documented and functional. The code must be in Python, but you can decide if to use Jupyter notebooks or not. I should be able to run the code to test it.
 3. A **beamer** (possibly in Latex) presentation for a 30-minute seminar.
- It is mandatory to present the project work in a seminar.

2 Course Outline

Lecture 1: Brownian Motions and Geometric BMs

- Brownian Motion (BM) and Geometric Brownian Motion (GBMs)
- Simulations of BMs, GBMs and Correlated GBMs

Lecture 2 : Numerical Methods for SDEs

1. Discretization Methods for SDEs: Euler-Maruyama Method; Milstein Method
2. SDE Simulations of: Ornstein–Uhlenbeck process; Cox–Ingersoll–Ross (CIR) process; Heston Model

Lecture 3: Pricing

1. Real-World Measure $\mathbb{P}$ vs Risk-Free Measure $\mathbb{P}$
2. Black Scholes Equation and Feynman–Kac Theorem
3. Monte Carlo Simulation: Generation of Stochastic Paths; Applications in Option Valuation

Lecture 4: Advanced Topics in Energy and Hints on Machine Learning

1. Energy markets
2. Machine Learning for Finance
3. Reinforcement Learning
4. Example Project: Application of Reinforcement Learning in Energy Markets.
5. Assignment of Projects
 - ◇ First week
Thursday 08/01/2026 10:30-12:30 - room I
 - ◇ Second week
Thursday 15/01/2026 10:30-12:30 - room I
 - ◇ Third week
Thursday 22/01/2026 10:30-12:30 - room I
 - ◇ Fourth week
Thursday 29/01/2025 10:30-12:30 - room I

References

- [1] C.W. Oosterlee and L.A. Grzelak - *Mathematical Modeling and Computation in Finance: With Exercises and Python and MATLAB Computer Codes*, World Scientific Publishing, 2019.
- [2] *Machine Learning in Finance: From Theory to Practice* by Matthew F. Dixon , Igor Halperin , Paul Bilokon. Springer (2020). <https://link.springer.com/book/10.1007/978-3-030-41068-1>

3 List of Potential Projects

1. Rough Volatility: Modelling and Pricing with Fractional Brownian Motion
2. Approximation of Iterated Stochastic Integrals and Rough Path Applications
3. Signature Methods for Financial Time Series Analysis and Trading
4. Neural Operators for Solving High-Dimensional PDEs in Finance
5. Deep Learning Methods for solving BSDEs
6. Entropic Risk Minimization for Multi-asset Portfolio Optimization via Neural Networks
7. Stochastic Volatility Models besides Heston Model (e.g. Local Volatility Model)
8. Simulation of Jump Processes (Lévy Processes) and Feynman-Kac theorem for jump diffusion process (Section 5 of [1])
9. Energy Markets: Modeling and Pricing Carbon Credits with Jump Diffusion
10. FinTech: Reinforcement Learning for Optimal Trading Strategies

Projects Details

1. Rough Volatility: Modelling and Pricing with Fractional Brownian Motion

- **Objective:** Explore and implement "rough" volatility models, where the volatility process is driven by a fractional Brownian motion (fBm) with a Hurst exponent $H < 0.5$. This project aims to simulate rough volatility paths and investigate their impact on option pricing, particularly the ability to reproduce the observed steep implied volatility skew for short-dated options.
- **Mathematical Framework:** Start with the rough Bergomi model:

$$dS_t = rS_t dt + \sqrt{v_t} S_t dW_t$$
$$v_t = \xi_0(t) \exp \left(\eta \tilde{W}_t - \frac{1}{2} \eta^2 t^{2H} \right)$$

where $\tilde{W}_t$ is a Riemann-Liouville fractional Brownian motion (correlated with W_t) with Hurst parameter $H \approx 0.1$. The key challenge is the efficient simulation of the fBm.

- **Coding Reference:** Python for simulation. Use libraries like `stochvol` or implement the hybrid scheme for fBm (Bennedsen, Lunde, Pakkanen 2017). Standard Monte Carlo methods can then be used for pricing.
- **Paper Reference:**
 - *Pricing under rough volatility* by Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum (2018).

- *The Volatility Surface: A Practitioner’s Guide* by Jim Gatheral and Nassim Taleb (Chapter on Roughness).

2. Approximation of Iterated Stochastic Integrals and Rough Path Applications

- **Objective:** Explore numerical approximation methods for iterated stochastic integrals, such as the Lévy area

$$\iint dB dB,$$

which arise in Lyons’ rough path theory. Investigate their role in simulating rough paths and solving rough differential equations, with applications to finance (e.g. rough volatility models).

- **Mathematical Framework:** Iterated integrals extend Itô/Stratonovich integration to higher order. Lyons’ theory enhances Brownian motion to a rough path $(B_t, \iint dB dB)$, enabling well-posedness of rough SDEs. Numerical methods include piecewise-linear Brownian interpolation, Riemann-type approximations, and advanced algorithms such as Wiktorsson’s method for Lévy area simulation.
- **Coding Reference:** Implement Brownian motion and Lévy area approximations in Python. Compare naïve discretizations with more advanced schemes (Wiktorsson’s algorithm, Fourier expansion methods). Numerical experiments should test convergence and computational cost.
- **Paper Reference:**
 - Lyons, T. (1998). *Differential equations driven by rough signals*.
 - Gatheral, Jaisson, Rosenbaum (2018). *Pricing under rough volatility*.
 - Wiktorsson, M. (2001). *Joint characteristic function and simultaneous simulation of iterated Itô integrals*.

3. Signature Methods for Financial Time Series Analysis and Trading

- **Objective:** Investigate the use of the Signature Transform, a tool from rough path theory, as a feature extraction method for financial time series. The project involves using signatures to create a parsimonious set of features for tasks like time series classification (e.g., regime detection) or as input to a trading model.
- **Mathematical Framework:** The signature of a path $X = (X_t)_{t \in [0, T]}$ is a collection of all its iterated integrals. The truncated signature up to level N provides a faithful feature set for the path. For a price path S_t , one can consider the path of log-returns $X_t = (\log(S_t), t)$ and compute its signature to capture its complex temporal dynamics in a linearizable feature space.
- **Coding Reference:** The `esig` and `signatory` (PyTorch) Python packages are specifically designed for calculating signatures quickly and efficiently. These features can then be fed into a simple linear model or a neural network.
- **Paper Reference:**

- *A Primer on the Signature Method in Machine Learning* by Ilya Chevyrev and Andrey Kormilitzin (2016).
- *Signature-based Portfolio Optimisation* by Blanka Horvath, Avi Mallia, et al. (2021).

4. Neural Operators for Solving High-Dimensional PDEs in Finance

- **Objective:** Implement and test Neural Operators, a deep learning architecture designed to learn mappings between function spaces, for solving high-dimensional partial differential equations (PDEs) that arise in finance, such as the Hamilton-Jacobi-Bellman (HJB) equation for optimal investment.
- **Mathematical Framework:** Unlike PINNs which solve a single instance of a PDE, Neural Operators (e.g., Fourier Neural Operators - FNOs) learn the *solution operator* of a family of PDEs. Given a parameter (e.g., initial condition, volatility function), the operator directly outputs the solution. The goal is to train an FNO to solve the multi-asset Black-Scholes PDE or a complex HJB equation for a range of parameters.
- **Coding Reference:** PyTorch or TensorFlow. The implementation would be based on the original FNO paper's code. Repository: https://github.com/zongyi-li/fourier_neural_operator
- **Paper Reference:**
 - *Fourier Neural Operator for Parametric Partial Differential Equations* by Zongyi Li, Nikola Kovachki, et al. (2020).
 - *Learning the solution operator of parametric partial differential equations with physics-informed DeepONets* by Lu et al. (2021). (Covers the related DeepONet architecture).

5. Deep Learning Methods for SDEs and Backward SDEs (BSDEs)

- **Objective:** Implement deep learning models for solving BSDEs, used to represent complex financial derivatives.
- **Mathematical Framework:** Given a BSDE of the form

$$Y_t = \xi + \int_t^T f(s, Y_s, Z_s) ds - \int_t^T Z_s dW_s$$

approximate the solution (Y, Z) with neural networks.

- **Coding Reference:** PyTorch or TensorFlow for implementing the deep BSDE solver, with forward-backward neural networks.
- **Paper Reference:**
 - “Deep BSDEs for High-Dimensional PDEs and Their Applications” by E et al. (2017)
 - “Backward Stochastic Differential Equations and Optimal Control” by Pham (2009).
 - “Backward Stochastic Differential Equations in Finance” by N. El Karoui, S. Peng, M. C. Quenez (2002)

6. Entropic Risk Minimization for Multi-asset Portfolio Optimization via Neural Networks

- **Objective:** Investigate entropic risk minimization techniques for optimizing a multi-asset portfolio by leveraging neural networks. The project aims to integrate deep learning methodologies to enhance portfolio management under risk constraints.
- **Mathematical Framework:** Formulate the portfolio optimization problem with an entropic risk measure, defined as:

$$\rho(X) = \frac{1}{\eta} \log \mathbb{E} [e^{-\eta X}] ,$$

where η represents the risk aversion parameter, and X is the portfolio return. Use neural networks to approximate optimal portfolio weights by learning the underlying distributional properties and minimizing the entropic risk measure across multiple assets.

- **Coding Reference:** Employ PyTorch or TensorFlow for developing neural network architectures to model entropic risk measures and perform portfolio optimization. Consider leveraging libraries such as PyPortfolioOpt for basic portfolio calculations and build custom neural architectures to address multi-asset optimization challenges.

Repository: <https://github.com/robertmartin8/PyPortfolioOpt>

- **Paper Reference:**
 - Entropic Risk Minimization and Portfolio Selection by *Hirschberger, Steuer, and Utz*. European Journal of Operational Research, 2007. <https://doi.org/10.1016/j.ejor.2005.12.017>
 - Deep Learning for Finance: Deep Portfolios by *Huang, Zhong, and Chen*. Journal of Financial Data Science, 2020. <https://jfds.pm-research.com/content/2/2/28>
 - “Neural Networks for Portfolio Management” by Heaton et al. (2017).

7. Stochastic Volatility Models (Beyond Heston Model)

- **Objective:** Implement models such as the Local Volatility Model

$$\begin{aligned} dS_t &= rS_t dt + b(V_t) \sigma_{LSV}(S_t, t) S_t dW_t, \\ dV_t &= a(V_t, t) dt + c(V_t, t) dZ_t, \\ dW_t dZ_t &= \rho dt. \end{aligned}$$

or the Stochastic Alpha Beta Rho (SABR) model, which are alternatives to Heston for stochastic volatility.

- **Mathematical Framework:** The Local Volatility Model derives volatility as a deterministic function of stock price and time, while the SABR model represents volatility with another SDE.
- **Coding Reference:** QuantLib for stochastic modeling or custom simulations.
- **Paper Reference:** “A Local Volatility Model” by Dupire (1994) and “The SABR Model in Illiquid Markets” by Hagan et al. (2002).

8. Simulation of Jump Processes (Lévy Processes) and Pricing

- **Objective:** Simulate jump-diffusion processes (e.g., Merton model) and price derivatives in models incorporating jumps, like the Kou and Variance Gamma models.
- **Mathematical Framework:** Start with jump-diffusion SDEs,

$$dX_t = \mu dt + \sigma dW_t + dJ_t$$

, where J_t is a jump process (e.g., Poisson-driven).

- **Coding Reference:** Python packages like NumPy and SciPy for simulations; PyMC for stochastic processes.
- **Paper Reference:** “Option Pricing when Jump-Diffusion Processes are Used” by Merton (1976).

9. Energy Markets: Modeling and Pricing Carbon Credits with Jump Diffusion

- **Objective:** Model and price carbon credits in energy markets, incorporating jumps due to policy changes or regulatory shifts.
- **Mathematical Framework:** Use jump-diffusion models or Markov-modulated jump processes to simulate carbon price dynamics.
- **Coding Reference:** Python libraries like QuantLib or custom implementations for pricing.
- **Paper Reference:** “The Pricing of Emission Allowances in the Presence of Market Frictions” by Carmona et al. (2009).

10. FinTech: Reinforcement Learning for Optimal Trading Strategies

- **Objective:** Develop reinforcement learning (RL) algorithms for optimal trading strategies.
- **Mathematical Framework:** Formulate as a Markov Decision Process, where the RL agent maximizes reward by learning an optimal trading policy.
- **Coding Reference:** Use RL libraries such as Stable-Baselines3 (Python) for implementing trading strategies.
- **Paper Reference:** “Deep Reinforcement Learning in High Frequency Trading” by Zhang et al. (2020).